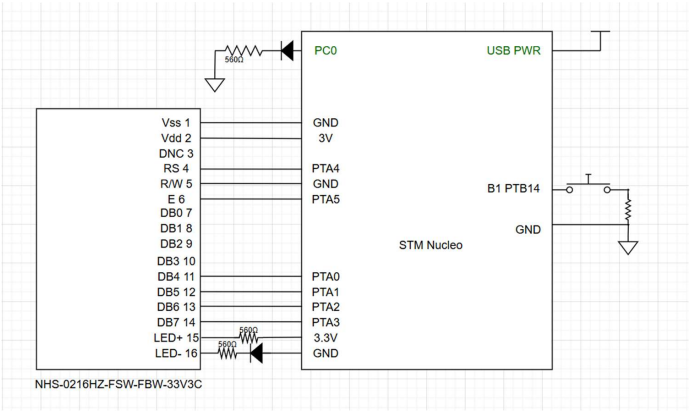
Maddie Masiello
Angel Soto

Lab Group A4
EE 329-03 S'25
2025-Apr-30

Introduction:

Link to YouTube Video Presentation: https://youtu.be/AYW05r9xL4c
(Date of posted video: 4/30/25)
(Length: 1:57)

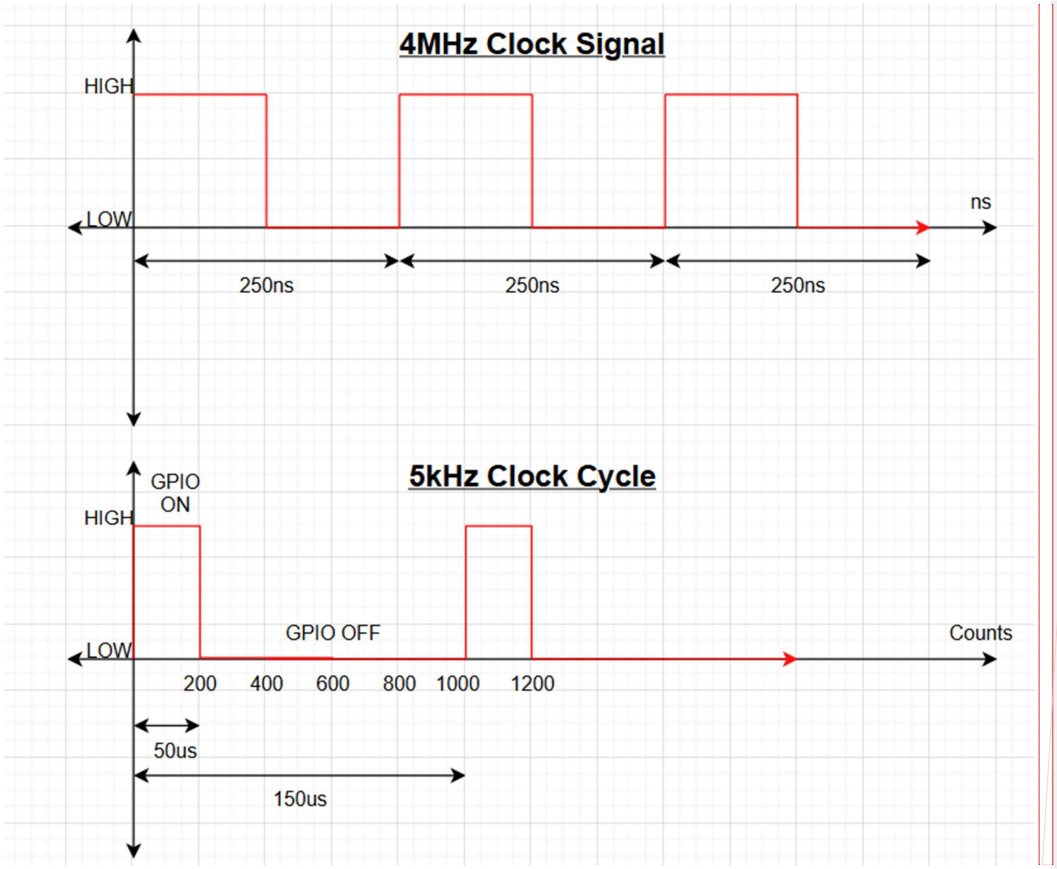


Obtained Data:
Part A: [ARR/CCR freq.-period-reload calculations, machine-drawn square-wave sketch]

Calculations:

Output Goal – frequency_out = 5kHz (which also means period_out = 200 microseconds) w/ 25% duty cycle.
Clock frequency (f_clk) = 4MHz

Period counts = f_clk * period_out = (4*10^6) * (200*10^-6) = 800 counts.
25% of 800 is 200.
Therefore, ARR = 799 and CCR1 = 200.



Commented [MM1]: need to not make this hand drawn

Screen Captures:

Figure A4(a): 5kHz with 25% Duty Cycle Square Wave

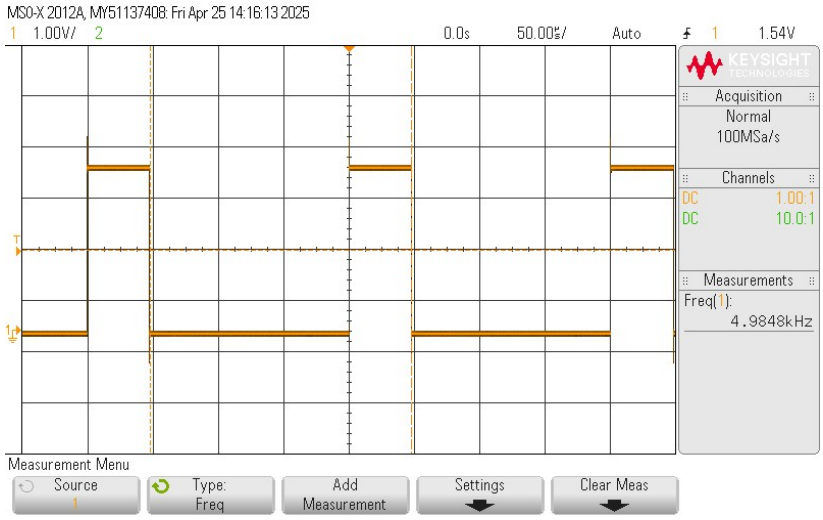


Figure A4(b): ISR Execution timing with MC0 Clock. Oscilloscope capture of PC0 (5 kHz output waveform) and PC1 (ISR timing pulse). PC1 briefly pulses high during each TIM2 interrupt (CCR1 compare match and ARR overflow), indicating ISR execution timing relative to the 5 kHz signal.

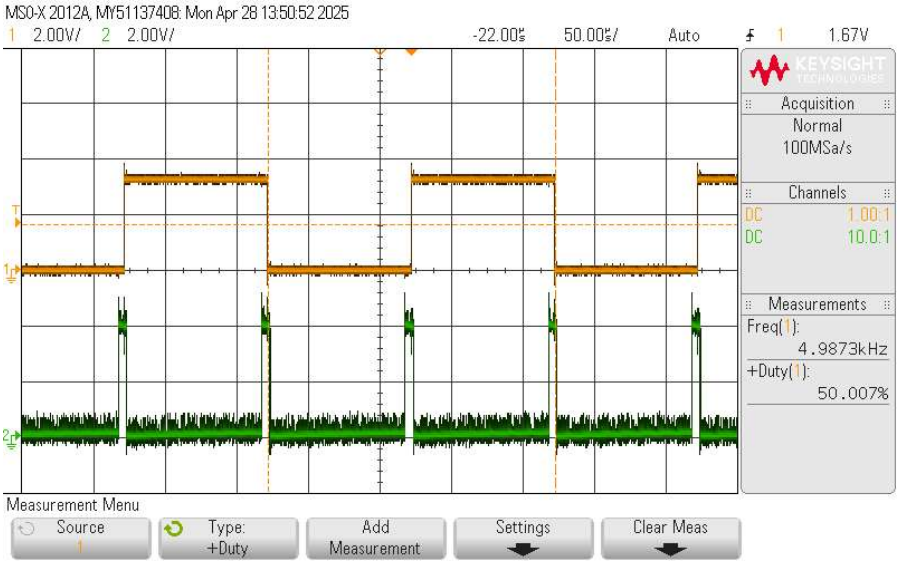
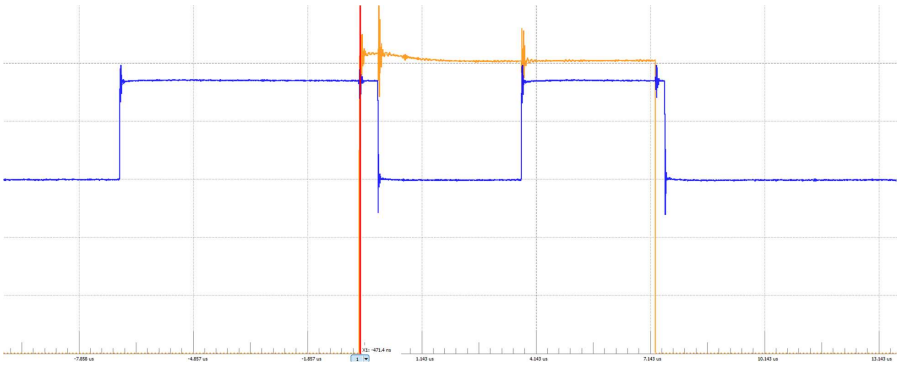


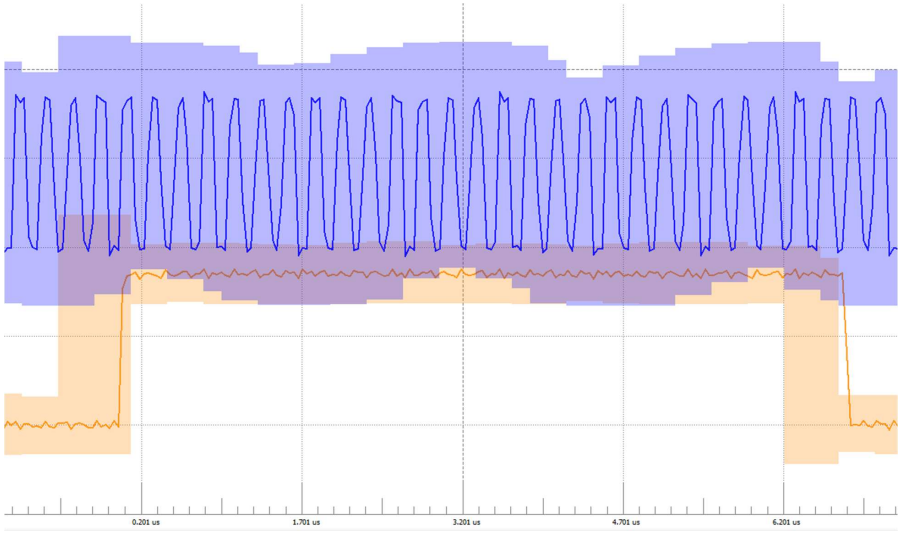
Figure A4(c): Part C - Smallest CCR1 value output. Oscilloscope capture of one PC0 square wave and PC1 ISR execution timing at CCR1 = 43. This is the minimum CCR1 value where proper toggling is still

observed before the signal "breaks." The CCR1 minimum value closely correlates to the number of clock cycles needed for the ISR to execute.



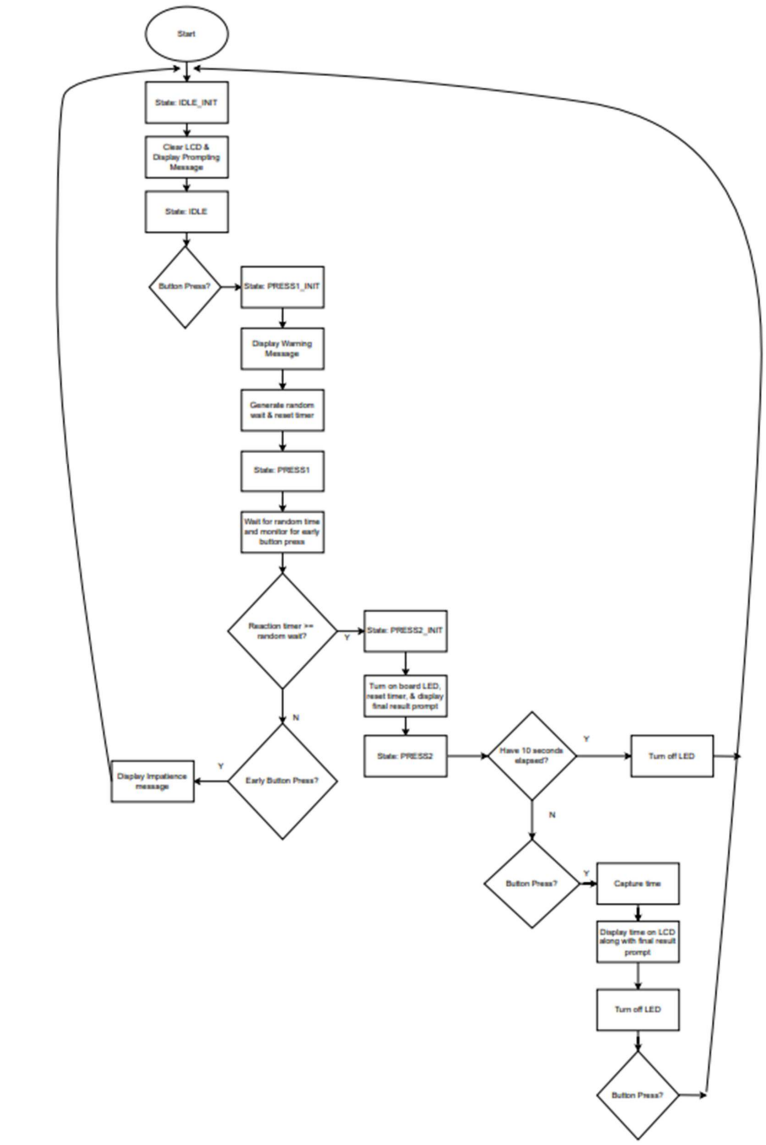
Observations and answers to questions ：

Part B – Number of MCO clock cycles for ISR: Zoomed-in oscilloscope capture of PC1 ISR timing pulse (orange) and PA8 4 MHz MCO clock (blue). The ISR execution time spans approximately 19 clock cycles.



Part C – Shortest CCR1 pulse

Part D – State Transition Diagram of game flow [*extra credit*]
https://drive.google.com/file/d/1IZzvBNhZHlAZgfDmooKRDRMptQVi2iST/view?usp=sharing


Start
State: IDLE_INIT
Clear LCD & Display Prompting Message
State: IDLE
Button Press?
State: PRESS1_INIT
Display Warning Message
Generate random wait & reset timer
State: PRESS1
Wait for random time and monitor for early button press
Reaction timer >= random wait?
Y
State: PRESS2_INIT
Turn on board LED, reset timer, & display final result prompt
N
Display Impatience message
Y
Early Button Press?
State: PRESS2
Have 10 seconds elapsed?
Y
Turn off LED
N
Button Press?
Y
Capture time
Display time on LCD along with final result prompt
Turn off LED
Button Press?

Source Code:

Appendix 1: Part B – 5 kHz, 50% duty cycle plus ISR timing bit & MC0 output

---
main.h

```
/******************************************************************************
 * EE 329 A4
 ******************************************************************************
 * @file           : main.h
 * @brief          : keypad configuration and debounced detection of keypresses
 * project         : EE 329 S'25 Assignment A4
 * authors         : Maddie Masiello, Angel Soto
 * version         : 1
 * date            : 230413
 * compiler        : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2023 STMicroelectronics.  All rights reserved.
 ******************************************************************************
 *
 ******************************************************************************
 */

 #ifndef __MAIN_H
 #define __MAIN_H

 #ifdef __cplusplus
 extern "C"
 {
 #endif

 /* Includes ------------------------------------------------------------------*/
 #include "stm32l4xx_hal.h"

     /* Exported functions prototypes ---------------------------------------------*/
     void SystemClock_Config(void);
     void Error_Handler(void);

 #ifdef __cplusplus
 }
 #endif
 #endif
```

---
main.c

```
/* USER CODE BEGIN Header */
/******************************************************************************
```

```
 * EE 329 A4
 ******************************************************************************
 * @file           : main.c
 * @brief          : Main program body, configures GPIO and TIM2 for PWM output
 * project         : EE 329 S'25 Assignment A4
 * authors         : Maddie Masiello - mmasiell@calpoly.edu, Angel Soto
 * version         : 1.0
 * date            : 2025-04-19
 * compiler        : STM32CubeIDE v1.12.0
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2025 STMicroelectronics. All rights reserved.
 ******************************************************************************
 * OVERVIEW:
 * This project is part B of EE 329 A4, which involves configuring GPIO pins
 * and TIM2 for PWM output. The program sets up GPIOC pin 0 as an output for an
 * LED and configures TIM2 to generate a PWM signal with a 25% duty cycle.
 ******************************************************************************
 * PINOUT: Uses A3 LCD pinout for LCD display
 * LED: PC0 (output)
 ******************************************************************************/
/* USER CODE END Header */

#include "main.h"
#include "lcd.h"
#include "delay.h"

/******************************************************************************
 * TIMER CONFIGURATION:
 * - TIM2 is configured for PWM output on PC0 with a 25% duty cycle.
 * - The timer is set to count up to 7999, which corresponds to a 200us period
 *   at a 4 MHz clock frequency.
 * T = 1/5000 = 200us,
 * Period counts = (fclock * Tout) = (4MHz * 200us) = 800 counts
 * High time = 0.25 * 800 = 200 counts = 50us
 * Low time = 0.75 * 800 = 600 counts = 150us
 * ARR = 800-1 = 799
 ******************************************************************************/
#define PERIOD 799      // 200 us period (4 MHz / 800)
#define DUTY   400      // 50% duty
#define OUT_PIN 0       // PC0 for LED
#define OUT_PORT GPIOC  // port C for LED

void SystemClock_Config(void); // Configure system clock

/******************************************************************************
 * setup_PC0(): Configures GPIOC pin 0 as output for LED
 * - Enables GPIOC clock, sets mode to output, type to push-pull,
 *   speed to high, and pull-up/pull-down to none.
 * - Initializes the pin to LOW state.
```

```
 ***************************************************************************/
void setup_PC0(void) {
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOCEN;      // Enable GPIOC clock
    GPIOC->MODER &= ~(0b11 << (2 * OUT_PIN)); // Reset mode
    GPIOC->MODER |=  (0b01 << (2 * OUT_PIN)); // Output mode
    GPIOC->OTYPER &= ~(1 << OUT_PIN);         // Push-pull
    GPIOC->OSPEEDR |= (0b11 << (2 * OUT_PIN));// High speed
    GPIOC->PUPDR &= ~(0b11 << (2 * OUT_PIN)); // No pull
    GPIOC->BRR = (1 << OUT_PIN);              // Start LOW
}

/***************************************************************************
 * setup_PC1(): Configures GPIOC pin 1 as output for LED
 * - Enables GPIOC clock, sets mode to output, type to push-pull,
 *  speed to high, and pull-up/pull-down to none.
 * - Initializes the pin to LOW state.
 ***************************************************************************/
void setup_PC1(void) {
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOCEN;   // Enable GPIOC clock (already enabled, safe)
    GPIOC->MODER &= ~(0b11 << (2 * 1));    // Reset mode for PC1
    GPIOC->MODER |=  (0b01 << (2 * 1));    // Set PC1 to output
    GPIOC->OTYPER &= ~(1 << 1);            // Push-pull
    GPIOC->OSPEEDR |= (0b11 << (2 * 1));   // High speed
    GPIOC->PUPDR &= ~(0b11 << (2 * 1));    // No pull
    GPIOC->BRR = (1 << 1);                 // Start LOW
}

/***************************************************************************
 * setup_TIM2(): Configures TIM2 for PWM output on PC0
 * - Enables TIM2 clock, sets up timer for PWM mode, and configures interrupt.
 *  sets CCR1 to enable the interupt at 25% duty cycle.
 * - Sets the timer period and duty cycle for the PWM signal.
 ***************************************************************************/
void setup_TIM2( int iDutyCycle ) {
    RCC->APB1ENR1 |= RCC_APB1ENR1_TIM2EN;           // enable clock for TIM2
    TIM2->DIER |= (TIM_DIER_CC1IE | TIM_DIER_UIE);  // enable event gen, rcv CCR1
    TIM2->ARR = 0xFFFFFFFF;                         // ARR = T = counts @4MHz
    TIM2->CCR1 = iDutyCycle;                        // ticks for duty cycle
    TIM2->SR &= ~(TIM_SR_CC1IF | TIM_SR_UIF);       // clr IRQ flag in status reg
    NVIC->ISER[0] |= (1 << (TIM2_IRQn & 0x1F));     // set NVIC interrupt: 0x1F
    __enable_irq();                                 // global IRQ enable
    TIM2->CR1 |= TIM_CR1_CEN;                       // start TIM2 CR1
 }

 /***************************************************************************
 * TIM2_IRQHandler(): Interrupt handler for TIM2
 * - Handles timer interrupts for compare match and overflow.
 * - Sets the output pin HIGH on compare match and LOW on overflow.
 * - Clears the interrupt flags.
 ***************************************************************************/
```

```c
void TIM2_IRQHandler(void) {
    GPIOC->BSRR = (GPIO_PIN_1); //PArt B: set PC1 HIGH at start of ISR

    if (TIM2->SR & TIM_SR_CC1IF) {//if TIM2 compare match
        TIM2->SR &= ~TIM_SR_CC1IF;// Clear interrupt flag
        GPIOC-> ODR ^= GPIO_ODR_OD0;
        TIM2->CCR1 += DUTY;
        GPIOC->BRR = (GPIO_PIN_1); // PC1 LOW at end of ISR
    }
    if (TIM2->SR & TIM_SR_UIF) {//if TIM2 overflow
        TIM2->SR &= ~TIM_SR_UIF;// Clear interrupt flag
        GPIOC->BRR = (GPIO_PIN_1); // PC1 LOW at end of ISR
    }
}

/*******************************************************************************
 * setup_MCO_CLK(): Configures MCO output on PA8
 * - Enables MCO clock, sets MCO source to SYSCLK (MSI), and configures PA8
 *  as alternate function for MCO output.
 *******************************************************************************/
void setup_MCO_CLK(void) {
    // enable MCO, MCOSEL = 0b0001 to select SYSCLK = MSI (4 MHz source)
    RCC->CFGR = ((RCC->CFGR & ~(RCC_CFGR_MCOSEL)) | (RCC_CFGR_MCOSEL_0));
    // configure MCO output on PA8
    RCC->AHB2ENR   |=  (RCC_AHB2ENR_GPIOAEN);
    GPIOA->MODER   &= ~(GPIO_MODER_MODE8);      // clear MODER bits to set ...
    GPIOA->MODER   |=  (GPIO_MODER_MODE8_1);    // MODE = alternate function
    GPIOA->OTYPER  &= ~(GPIO_OTYPER_OT8);       // push-pull output
    GPIOA->PUPDR   &= ~(GPIO_PUPDR_PUPD8);      // pullup & pulldown OFF
    GPIOA->OSPEEDR |=  (GPIO_OSPEEDR_OSPEED8);   // high speed
    GPIOA->AFR[1]  &= ~(GPIO_AFRH_AFSEL8);      // select MCO (AF0) on PA8 (AFRH)
 }


/*******************************************************************************
 * main(): Main function for the program
 * - Configures the system clock
 * - Sets up GPIO pins for LED output
 * - Configures TIM2 for PWM output on PC0
 *******************************************************************************/
int main(void) {
    HAL_Init();              // Basic system setup
    SystemClock_Config();    // Ensure MSI is set to 4 MHz
    setup_PC0();             // Configure PC0
    setup_PC1();             // Configure PC1
    setup_MCO_CLK();         // Configure MCO output on PA8
    setup_TIM2(DUTY);        // Enable TIM2 with 25% duty
    while (1);               // Sit in loop while timer runs
}
```

```c
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Configure the main internal regulator output voltage */
    if (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the RCC Oscillators according to the specified parameters
     * in the RCC_OscInitTypeDef structure. */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_MSI;
    RCC_OscInitStruct.MSIState = RCC_MSI_ON;
    RCC_OscInitStruct.MSICalibrationValue = 0;
    RCC_OscInitStruct.MSIClockRange = RCC_MSIRANGE_6;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_PCLK1
| RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_MSI;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
    {
        Error_Handler();
    }
}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
 * @brief  This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return
state */
```

```
    __disable_irq();
    while (1)
    {
    }
}

#ifdef USE_FULL_ASSERT
void assert_failed(uint8_t *file, uint32_t line)
{
}
#endif
```

---

lcd.h

```
/* USER CODE BEGIN Header */
/**************************************************************************
 * EE 329 A4
 **************************************************************************
 * @file           : lcd.h
 * @brief          : lcd.h header file for LCD display functions
 * project         : EE 329 S'25 Assignment A4
 * authors         : Maddie Masiello - mmasiell@calpoly.edu, Angel Soto
 * version         : 1.0
 * date            : 2025-04-19
 * compiler        : STM32CubeIDE v1.12.0
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2025 STMicroelectronics. All rights reserved.
 **************************************************************************
 * OVERVIEW:
 * This project is part B of EE 329 A4, which involves configuring GPIO pins
 * for the LCD display. The program sets up GPIOA pins 0-5 for the LCD control
 * and data lines. The LCD is initialized in 4-bit mode, and functions are
 * provided to send commands, write characters, and display strings.
 **************************************************************************
 * PINOUT:
 *
 * LED: PC0 (output)
 **************************************************************************/
/* USER CODE END Header */
#ifndef INC_LCD_H_
#define INC_LCD_H_

#include "stm32l4xx.h"
#include "delay.h"

#define LCD_PORT GPIOA
#define LCD_RS GPIO_PIN_4 // PA4
#define LCD_EN GPIO_PIN_5 // PA5
```

```h
#define LCD_DB4 GPIO_PIN_0 // PA0
#define LCD_DB5 GPIO_PIN_1 // PA1
#define LCD_DB6 GPIO_PIN_2 // PA2
#define LCD_DB7 GPIO_PIN_3 // PA3
#define LCD_DATA_BITS (LCD_DB4 | LCD_DB5 | LCD_DB6 | LCD_DB7)
#define LCD_PORTEN RCC_AHB2ENR_GPIOAEN // port enable mask
#define RSpos 4
#define ENpos 5
#define DB4pos 0
#define DB5pos 1
#define DB6pos 2
#define DB7pos 3


void LCD_init(void);
void LCD_command(uint8_t cmd);
void LCD_write_char(uint8_t letter);
void LCD_write_string(char *str);
void LCD_4b_command(uint8_t command);
void LCD_set_cursor(uint8_t pos[2]);
void LCD_clear(void);
void DisplayTime(uint8_t *digits);

#endif /* INC_LCD_H_ */
```

---

lcd.c

```c
#include "lcd.h"
/* USER CODE BEGIN Header */
/**************************************************************************
 * EE 329 A4
 **************************************************************************
 * @file           : lcd.c
 * @brief          : lcd.c source file for LCD display functions
 * project         : EE 329 S'25 Assignment A4
 * authors         : Maddie Masiello - mmasiell@calpoly.edu, Angel Soto
 * version         : 1.0
 * date            : 2025-04-19
 * compiler        : STM32CubeIDE v1.12.0
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2025 STMicroelectronics. All rights reserved.
 **************************************************************************
 * OVERVIEW:
 * This file contains the implementation of functions for controlling an LCD
 * display. The functions include initialization, sending commands, writing
 * characters, and displaying strings. The LCD is configured to operate in
 * 4-bit mode, and the GPIO pins are set up accordingly. The functions also
 * include a delay function to ensure proper timing for the LCD commands.
```

```
 ******************************************************************************
 * PINOUT:
 * LED: PC0 (output)
 ******************************************************************************/
/* USER CODE END Header */

void LCD_GPIO_Config(void)
{
    RCC->AHB2ENR |= (LCD_PORTEN); //Enable LCD GPIO Register
    LCD_PORT->MODER &= ~(0b11<<(2*DB4pos) |
    0b11<<(2*DB5pos) |
    0b11<<(2*DB6pos) |
    0b11<<(2*DB7pos) |
    0b11<<(2*RSpos) |
    0b11<<(2*ENpos)); //reset
LCD_PORT->MODER |= (0b01<<(2*DB4pos) |
    0b01<<(2*DB5pos) |
    0b01<<(2*DB6pos) |
    0b01<<(2*DB7pos) |
    0b01<<(2*RSpos) |
    0b01<<(2*ENpos)); //output
LCD_PORT->OTYPER &= ~(0b1<<(DB4pos) |
    0b1<<(DB5pos) |
    0b1<<(DB6pos) |
    0b1<<(DB7pos) |
    0b1<<(RSpos) |
    0b1<<(ENpos)); //psh/pull
LCD_PORT->OSPEEDR |= (0b11<<(2*DB4pos) |
    0b11<<(2*DB5pos) |
    0b11<<(2*DB6pos) |
    0b11<<(2*DB7pos) |
    0b11<<(2*RSpos) |
    0b11<<(2*ENpos));//vhi spd
LCD_PORT->PUPDR &= ~(0b11<<(2*DB4pos) |
    0b11<<(2*DB5pos) |
    0b11<<(2*DB6pos) |
    0b11<<(2*DB7pos) |
    0b11<<(2*RSpos) |
    0b11<<(2*ENpos)); //reset
LCD_PORT->PUPDR |= (0b00<<(2*DB4pos) |
    0b00<<(2*DB5pos) |
    0b00<<(2*DB6pos) |
    0b00<<(2*DB7pos) |
    0b00<<(2*RSpos) |
    0b00<<(2*ENpos)); //nopupl
}


void LCD_init(void)
{
```

```
    LCD_GPIO_Config(); // configure GPIO pins for LCD

    delay_us(40000); // power-up wait 40 ms
    for (int idx = 0; idx < 3; idx++)
    {                          // wake up 1,2,3: DATA = 0011 XXXX
        LCD_4b_command(0x30); // HI 4b of 8b cmd, low nibble = X
        delay_us(200);
    }
    LCD_4b_command(0x20); // fcn set #4: 4b cmd set 4b mode - next 0x28:2-line
    delay_us(40);         // remainder of LCD init removed - see LCD datasheets

    LCD_command(0x28); // Function set: 4-bit, 2-line, 5x8 font
    delay_us(300);

    LCD_command(0x10); // Set cursor move direction
    delay_us(300);

    LCD_command(0x0F); // Display ON, cursor ON, blink ON
    delay_us(300);

    LCD_command(0x06); // Entry mode: increment cursor, no shift
    delay_us(300);

    LCD_command(0x01); // Clear display
    delay_us(2000);    // Extra wait for clear command

    LCD_command(0x80); // Set cursor to home (DDRAM address 0)
    delay_us(300);
}

void LCD_pulse_ENA(void)
{
    // ENAble line sends command on falling edge
    // set to restore default then clear to trigger
    LCD_PORT->ODR |= (LCD_EN);  // ENABLE = HI
    delay_us(5);                // TDDR > 320 ns
    LCD_PORT->ODR &= ~(LCD_EN); // ENABLE = LOW
    delay_us(5);                // low values flakey, see A3:p.1
}

void LCD_4b_command(uint8_t command)
{
    // LCD command using high nibble only - used for 'wake-up' 0x30 commands
    LCD_PORT->ODR &= ~(LCD_DATA_BITS); // clear DATA bits
    LCD_PORT->ODR |= (command & 0x0F); // changed: place nibble directly on PA0-PA3
    delay_us(5);
    LCD_pulse_ENA();
}

void LCD_command(uint8_t command)
```

```
{
    // send command to LCD in 4-bit instruction mode
    // HIGH nibble then LOW nibble, timing sensitive
    LCD_PORT->ODR &= ~(LCD_DATA_BITS);                 // isolate cmd bits
    LCD_PORT->ODR |= ((command >> 4) & LCD_DATA_BITS); // HIGH shifted low
    delay_us(5);
    LCD_pulse_ENA(); // latch HIGH NIBBLE

    LCD_PORT->ODR &= ~(LCD_DATA_BITS);          // isolate cmd bits
    LCD_PORT->ODR |= (command & LCD_DATA_BITS); // LOW nibble
    delay_us(5);
    LCD_pulse_ENA(); // latch LOW NIBBLE
}

void LCD_write_char(uint8_t letter)
{
    // calls LCD_command() w/char data; assumes all ctrl bits set LO in LCD_init()
    LCD_PORT->ODR |= (LCD_RS); // RS = HI for data to address
    delay_us(400);
    LCD_command(letter);        // character to print
    LCD_PORT->ODR &= ~(LCD_RS); // RS = LO
}

void LCD_write_string(char *str)
{
    // writes each character in a null-terminated string to LCD
    while (*str)
    {
        LCD_write_char(*str++);
    }
    delay_us(400);
}

void LCD_clear(void)
{
    // clears LCD display and sets cursor to home position
    LCD_command(0x01); // clear display command
    delay_us(10000);   // wait for clear command to complete
}

void LCD_set_cursor(uint8_t pos[2])
{
    // sets LCD cursor to given (col, row)
    uint8_t row_offsets[] = {0x00, 0x40};
    uint8_t addr = 0x80 | (pos[0] + row_offsets[pos[1]]);
    LCD_command(addr);
}

/*******************************************************************************
 *  DisplayTime(): Displays the current time on the LCD in MM:SS format
```

```
 * - Takes an array of 4 digits and displays them on the LCD.
 ******************************************************************************/
void DisplayTime(uint8_t *digits)
{ // points to the digits array

    // sets cursor to the appropriate position on the lcd and display the digits
    LCD_set_cursor((uint8_t[]){11, 1});
    LCD_write_char(digits[0] + '0');
    LCD_set_cursor((uint8_t[]){12, 1});
    LCD_write_char(digits[1] + '0');
    LCD_set_cursor((uint8_t[]){13, 1});
    LCD_write_char(':');
    LCD_set_cursor((uint8_t[]){14, 1});
    LCD_write_char(digits[2] + '0');
    LCD_set_cursor((uint8_t[]){15, 1});
    LCD_write_char(digits[3] + '0');
}
```

Appendix 2: User Reaction Timer Application

---

main.h

```
/******************************************************************************
 * EE 329 A4 - D
 ******************************************************************************
 * @file           : main.h
 * @brief          : Reaction timer Game
 * project         : EE 329 S'25 Assignment A4 - Part D
 * authors         : Maddie Masiello - mmasiell@calpoly.edu
 *                          : Angel Soto - asoto36@calpoly.edu
 * version         : 1.0
 * date            : 2025-04-19
 * compiler        : STM32CubeIDE v1.12.0
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2025 STMicroelectronics. All rights reserved.
 ******************************************************************************
 * OVERVIEW:
 * This project is the header file for main.c.
 ******************************************************************************
 * PINOUT:
 *
 ******************************************************************************/

/* Define to prevent recursive inclusion -------------------------------------*/
#ifndef __MAIN_H
#define __MAIN_H

#ifdef __cplusplus
extern "C" {
#endif

/* Includes ------------------------------------------------------------------*/
#include "stm32l4xx_hal.h"
```

```c
/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Exported types ------------------------------------------------------------*/
/* USER CODE BEGIN ET */

/* USER CODE END ET */

/* Exported constants --------------------------------------------------------*/
/* USER CODE BEGIN EC */

/* USER CODE END EC */

/* Exported macro ------------------------------------------------------------*/
/* USER CODE BEGIN EM */

/* USER CODE END EM */

/* Exported functions prototypes ---------------------------------------------*/
void Error_Handler(void);

/* USER CODE BEGIN EFP */

/* USER CODE END EFP */

/* Private defines -----------------------------------------------------------*/
#define B1_Pin GPIO_PIN_13
#define B1_GPIO_Port GPIOC
#define LD3_Pin GPIO_PIN_14
#define LD3_GPIO_Port GPIOB
#define USB_OverCurrent_Pin GPIO_PIN_5
#define USB_OverCurrent_GPIO_Port GPIOG
#define USB_PowerSwitchOn_Pin GPIO_PIN_6
#define USB_PowerSwitchOn_GPIO_Port GPIOG
#define STLK_RX_Pin GPIO_PIN_7
#define STLK_RX_GPIO_Port GPIOG
#define STLK_TX_Pin GPIO_PIN_8
#define STLK_TX_GPIO_Port GPIOG
#define USB_SOF_Pin GPIO_PIN_8
#define USB_SOF_GPIO_Port GPIOA
#define USB_VBUS_Pin GPIO_PIN_9
#define USB_VBUS_GPIO_Port GPIOA
#define USB_ID_Pin GPIO_PIN_10
#define USB_ID_GPIO_Port GPIOA
#define USB_DM_Pin GPIO_PIN_11
#define USB_DM_GPIO_Port GPIOA
#define USB_DP_Pin GPIO_PIN_12
#define USB_DP_GPIO_Port GPIOA
#define TMS_Pin GPIO_PIN_13
#define TMS_GPIO_Port GPIOA
#define TCK_Pin GPIO_PIN_14
#define TCK_GPIO_Port GPIOA
#define SWO_Pin GPIO_PIN_3
#define SWO_GPIO_Port GPIOB
#define LD2_Pin GPIO_PIN_7
#define LD2_GPIO_Port GPIOB

/* USER CODE BEGIN Private defines */
```

```
/* USER CODE END Private defines */

#ifdef __cplusplus
}
#endif

#endif /* __MAIN_H */
```

---

main.c

```
/* USER CODE BEGIN Header */
/**************************************************************************
 * EE 329 A4 - D
 **************************************************************************
 * @file           : main.c
 * @brief          : Reaction Timer Game
 * project         : EE 329 S'25 Assignment A4 - Part D
 * authors         : Maddie Masiello - mmasiell@calpoly.edu
 *                 : Angel Soto – asoto36@calpoly.edu
 * version         : 1.0
 * date            : 2025-04-19
 * compiler        : STM32CubeIDE v1.12.0
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2025 STMicroelectronics. All rights reserved.
 **************************************************************************
 * OVERVIEW:
 * This project is the main file for the Reaction Timer
 **************************************************************************
 * PINOUT:
 *
 **************************************************************************/
/* USER CODE END Header */

#include "main.h"
#include "lcd.h"
#include "delay.h"

/**************************************************************************
 * TIMER CONFIGURATION:
 * - TIM2 is
 **************************************************************************/
#define BUTTON_PIN GPIO_PIN_13 // Button pin (PC13)
#define OUT_PORT GPIOB       // port B for LED/button
#define BUTTON_PORT GPIOC    // port C for button
#define PERIOD 0xFFFFFFFF    // max timer value for 32-bit timer
#define BOARD_LED GPIO_PIN_7   // PB7 for board LED
#define MAX_WAIT_TIME 10000000 // 10s max wait time
#define TIM2_TICK_US 250U    // 250 microseconds per tick

/**************************************************************************
 * State Descriptions:
 * IDLE:
 *  - Wait for first button press
 *  - LCD displays:
 *      "EE329 A4 REACT"
 *      "PUSH SW TO TRIG"
```

```
 *  - If first button pressed -> transition to PRESS1
 *
 * PRESS1:
 *  - LCD counts up
 *  - 250 microsecond accuracy
 *  - Turn on board LED
 *  - If no button press after 10s -> return to IDLE
 *  - Else if button pressed -> transition to PRESS2
 *
 * PRESS2:
 *  - Capture timer value
 *  - Convert to decimal seconds (1ms resolution)
 *  - Display timer on LCD
 *  - If user presses button -> return to IDLE
 ******************************************************************************/
typedef enum
{
	IDLE_INIT,
	IDLE,
	IDLE_EXIT,
	PRESS1_INIT,
	PRESS1,
	PRESS1_EXIT,
	PRESS2_INIT,
	PRESS2,
	PRESS2_EXIT,
	HOLD,
	HOLD_EXIT

} State_t;

volatile State_t state = IDLE_INIT; // Initialize state variable

/******************************************************************************
 * Global Variables:
 * - reaction_timer_us: counts in microseconds
 * - button_pressed_flag: for main loop to see button presses
 * - lcd_initialized_flag: to avoid re-clearing LCD every loop
 * - random_wait_time_us: random wait time in microseconds
 ******************************************************************************/
volatile uint8_t button_pressed_flag = 0; // for main loop to see button presses
volatile uint32_t random_wait_time_us = 0;
volatile uint32_t reaction_timer_us = 0;

/******************************************************************************
 * setup_PC13(): Configures GPIOC pin 13 as input for button
 ******************************************************************************/
void setup_PC13(void)
{
	RCC->AHB2ENR |= RCC_AHB2ENR_GPIOCEN;
	GPIOC->MODER &= ~(0b11 << (2 * 13));
	GPIOC->PUPDR = (GPIOC->PUPDR & ~(0b11 << (2 * 13))) | (2U << (2 * 13));
}

/******************************************************************************
 * setup_PC7(): Configures GPIOC pin 7 as LED
 ******************************************************************************/
void setup_PB7(void)
{
	RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN; // Enable GPIOC clock
	GPIOB->MODER &= ~(0b11 << (2 * 7));     // Reset mode
```

```
	GPIOB->MODER |= (0b01 << (2 * 7));	// Output mode
	GPIOB->OTYPER &= ~(1 << 7);			// Push-pull
	GPIOB->OSPEEDR |= (0b11 << (2 * 7)); // High speed
	GPIOB->PUPDR &= ~(0b11 << (2 * 7));	// No pull
	GPIOB->BRR = (1 << 7);				// Start LOW
}

/*****************************************************************************
 * Button_Pressed(): Checks if the button is pressed
 * - Returns 1 if pressed, 0 otherwise.
 *****************************************************************************/
uint8_t Button_Pressed(void)
{
	if (GPIOC->IDR & (1U << 13))
	{
		delay_us(10000); // debounce
		if (GPIOC->IDR & (1U << 13))
			return 1; // pressed
	}
	return 0; // not pressed
}

/*****************************************************************************
 * setup_TIM2():
 *****************************************************************************/
void setup_TIM2(void)
{
	// 4 MHz / (PSC+1) = 1 MHz → tick = 1 µs
	// ARR = 250-1 → interrupt every 250 µs
	RCC->APB1ENR1 |= RCC_APB1ENR1_TIM2EN;
	TIM2->PSC = 4U - 1U;
	TIM2->ARR = 250U - 1U;
	TIM2->SR &= ~TIM_SR_UIF;
	TIM2->DIER |= TIM_DIER_UIE;
	NVIC->ISER[0] |= (1 << (TIM2_IRQn & 0x1F));
	TIM2->CR1 |= TIM_CR1_CEN;
}

/*****************************************************************************
 * TIM2_IRQHandler():
 * - Handles the TIM2 interrupt request
 * - Checks if the interrupt was triggered by an update event (timer overflow)
 * - If so, clears the interrupt flag and increments the reaction timer.
 * - If in PRESS1 state, increments the reaction timer by 250 microseconds.
 *****************************************************************************/
void TIM2_IRQHandler(void)
{
	if (TIM2->SR & TIM_SR_UIF)
	{
		TIM2->SR &= ~TIM_SR_UIF;
		reaction_timer_us += TIM2_TICK_US;
	}
}

/*****************************************************************************
 * RNG_Init(): Initializes the RNG peripheral
 * - Enables the RNG clock and sets the RNG to ready state.
 *****************************************************************************/
void RNG_Init(void)
{
	RCC->CRRCR |=    RCC_CRRCR_HSI48ON;                    // request HSI48
```

```
	while(!(RCC->CRRCR & RCC_CRRCR_HSI48RDY));		// wait until ready
	RCC->AHB2ENR |= RCC_AHB2ENR_RNGEN;
	RNG->CR |= RNG_CR_RNGEN;
}

/*************************************************************************
 * RNG_GetRandomNumber(): Generates a random number using the RNG peripheral
 * - Waits for the RNG to be ready and returns a random number.
 *************************************************************************/
 uint32_t RNG_GetRandomNumber(void)
{
	//return 0;  //TODO: changed to 0 for debugging
	while ((RNG->SR & RNG_SR_DRDY) == 0)
		;
	return RNG->DR;
}

 /*************************************************************************
  * System Clock Configuration
  *************************************************************************/
void SystemClock_Config(void)
{
	RCC_OscInitTypeDef RCC_OscInitStruct = {0};
	RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

	/** Configure the main internal regulator output voltage */
	if (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
	{
		Error_Handler();
	}

	/** Initializes the RCC Oscillators according to the specified parameters
	 * in the RCC_OscInitTypeDef structure. */
	RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_MSI;
	RCC_OscInitStruct.MSIState = RCC_MSI_ON;
	RCC_OscInitStruct.MSICalibrationValue = 0;
	RCC_OscInitStruct.MSIClockRange = RCC_MSIRANGE_6;
	RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
	if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
	{
		Error_Handler();
	}

	/** Initializes the CPU, AHB and APB buses clocks */
	RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK |
RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
	RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_MSI;
	RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
	RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
	RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

	if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
	{
		Error_Handler();
	}
}

/*************************************************************************
 * main():
 *************************************************************************/
int main(void)
```

```c
{
	HAL_Init();                 // Basic system setup
	SystemClock_Config(); // Ensure MSI is set to 4 MHz
	setup_PB7();  //Configure output (Blue) Board LED
	setup_PC13(); // Configure PC13
	setup_TIM2(); //Configure TIM2
	LCD_init(); // Initialize LCD
	RNG_Init();

	while (1)
	{
		switch (state)
		{
		/*****************************************************************************
		 * case IDLE:
		 * - Wait for first button press
		 *     - LCD displays:
		 *           "EE329 A4 REACT"
		 *           "PUSH SW TO TRIG"
		 *     - If first button pressed -> transition to PRESS1
		 *****************************************************************************/
		case IDLE_INIT:
			__disable_irq();
			LCD_clear();                                  // Clear LCD
			LCD_set_cursor((uint8_t[]){0, 0});     // Set cursor to first line
			LCD_write_string("EE329 A4 REACT"); // Display first line

			LCD_set_cursor((uint8_t[]){0, 1});      // Set cursor to second line
			LCD_write_string("PUSH SW TO TRIG"); // Display second line
			__enable_irq();

			state = IDLE; // Transition to IDLE state
			break;

		case IDLE:
			// if button pressed -> transition to PRESS1
			if (Button_Pressed())
			{
				state = IDLE_EXIT; // Transition to PRESS1_INIT state
			}

			break;

		case IDLE_EXIT:
			if (!Button_Pressed())
			{
				state = PRESS1_INIT; // Transition to PRESS1_INIT state
			}
			break;


		/*****************************************************************************
		 * case PRESS1:
		 * - Clear LCD and display "Ready?"
		 * - Generate random number between 2s and 8s
		 * - Start counting down from random number
		 * - If button pressed -> transition to PRESS2
		 * - If no button press after 10s -> return to IDLE
		 *****************************************************************************/
		case PRESS1_INIT:
```

```
        //Reset LCD
        __disable_irq();
        LCD_clear();

        //Prompting Message
        LCD_set_cursor((uint8_t[]){0, 0});
        LCD_write_string("PREPARE YOURSELF");
        LCD_set_cursor((uint8_t[]){4, 1}); //Centers final word on second line
        LCD_write_string("MORTAL!");
        __enable_irq();

        // Generate random wait time between 2 and 8 seconds
        random_wait_time_us = (RNG_GetRandomNumber() % 6000000) + 2000000; // 2-8s
        reaction_timer_us = 0; // grabbing Start time in ticks
        state = PRESS1; // Transition to PRESS1 state
        break;

case PRESS1:

        // Turn on LED and reaction timer if wait time has passed

        // calculating duration using current - start time
        //  if this time is >= randomly generated time, enter PRESS2_INIT
        if (reaction_timer_us >= random_wait_time_us)
        {
                state = PRESS2_INIT;
        }
        // Check if user pressed button too early (before LED)
        else if (Button_Pressed())
        {
                //Reset LCD
                __disable_irq();
                LCD_clear();

                //"Impatience Alert" Message
                LCD_set_cursor((uint8_t[]){3, 0});
                LCD_write_string("HOLD YOUR");
                LCD_set_cursor((uint8_t[]){4, 1});
                LCD_write_string("HORSES!"); //Centers final word on second line
                __enable_irq();
                delay_us(2000000); // 2 second pause
                state = IDLE_INIT; //Transition to IDLE_INIT state
        }

        break;


//add PRESS1_EXIT state
case PRESS1_EXIT:

        //Check for no button press to transition to PRESS2_INIT state
        if (!Button_Pressed())
        {
                state = PRESS2_INIT; // Transition to PRESS2_INIT state
        }
        break;



/*****************************************************************************
 * case PRESS2:
```

```
 * - Capture timer value
 * - Convert to decimal seconds (1ms resolution)
 * - Display timer on LCD
 * - If user presses button -> return to IDLE
 ******************************************************************************/
case PRESS2_INIT:

	//Reset LCD
	__disable_irq();
	LCD_clear();

	//Final Timing Message
	LCD_set_cursor((uint8_t[]){0, 0});
	LCD_write_string("REACTION TIME:");
	__enable_irq();

	GPIOB->BSRR = BOARD_LED; // turn ON PB7 LED
	reaction_timer_us = 0;
	state = PRESS2; // Transition to PRESS2 state
	break;

case PRESS2:
	// Check timeout (10s max)
	if (reaction_timer_us >= MAX_WAIT_TIME)
	{
		GPIOB->BRR = BOARD_LED; // turn OFF PB7 LED
		state = IDLE_INIT; //Transition to IDLE_INIT state
		break;
	}

	if (Button_Pressed())
	{
		// Convert the reaction time from microseconds to milliseconds
		// Multiply by 1000 to convert to nanoseconds, then divide by system clk
		uint32_t time_ms = (reaction_timer_us * 1000) / SystemCoreClock;

		// Break down the time into individual display components:
		uint8_t sec = time_ms / 1000;                    // Whole seconds
		uint8_t msec_hundreds = (time_ms % 1000) / 100;  // Hundreds place of ms
		uint8_t msec_tens = (time_ms % 100) / 10;        // Tens place of ms
		uint8_t msec_ones = (time_ms % 10);              // Ones place of ms

		// Temporarily disable interrupts while updating LCD to avoid corruption
		__disable_irq();
		LCD_set_cursor((uint8_t[]){0, 1}); //Set cursor to start of 2nd LCD line

		// Display seconds and decimal point
		if (sec == 0)
		{
			LCD_write_string("0."); // e.g., "0." if less than 1 second
		}
		else
		{
			LCD_write_char(sec + '0');  // Convert sec to ASCII character
			LCD_write_char('.');        // Add decimal point
		}

		 // Display fractional milliseconds as individual digits
		    LCD_write_char(msec_hundreds + '0'); // e.g., "5" in 0.523
		    LCD_write_char(msec_tens + '0');     // e.g., "2" in 0.523
		    LCD_write_char(msec_ones + '0');     // e.g., "3" in 0.523
```

```
                        LCD_write_string(" s");              // Add unit label " s"
                        __enable_irq(); // Re-enable interrupts after safe LCD update

                GPIOB->BRR = BOARD_LED; // turn OFF LED now

                state = PRESS2_EXIT; // Transition to HOLD state
            }

            break;

            case PRESS2_EXIT:

            //Check for no button press to transition to HOLD state
            if (!Button_Pressed())
            {
                state = HOLD; // Transition to HOLD state
            }
            break;

        case HOLD:

            // Check if button is pressed to return to IDLE
            if (Button_Pressed())
            {
                state = HOLD_EXIT; // Transition to IDLE state
            }
            break;

        case HOLD_EXIT:

            //Check for no button press to transition to IDLE state
            if (!Button_Pressed())
            {
                state = IDLE_INIT; // Transition to IDLE state
            }
            break;


        }
    }
}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
 * @brief  This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
}

#ifdef USE_FULL_ASSERT
```

```
void assert_failed(uint8_t *file, uint32_t line)
{
}
#endif
```

---

delay.h

```
/* USER CODE BEGIN Header */
/**************************************************************************
 * EE 329 A4
 **************************************************************************
 * @file           : delay.h
 * @brief          :
 * project         : EE 329 S'25 Assignment A4
 * authors         : Maddie Masiello - mmasiell@calpoly.edu
 *                 : Angel Soto - asoto36@calpoly.edu
 * version         : 1.0
 * date            : 2025-04-19
 * compiler        : STM32CubeIDE v1.12.0
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2025 STMicroelectronics. All rights reserved.
 **************************************************************************
 * OVERVIEW:
 * This project is the header file for delay.c.
**************************************************************************/
/* USER CODE END Header */


#ifndef INC_DELAY_H_
#define INC_DELAY_H_

#include "stm32l4xx.h"

void SysTick_Init(void);
void delay_us(uint32_t time_us);

#endif /* INC_DELAY_H_ */
```

---

delay.c

```
/* USER CODE BEGIN Header */
/**************************************************************************
 * EE 329 A4
 **************************************************************************
 * @file           : delay.c
 * @brief          :
 * project         : EE 329 S'25 Assignment A4
 * authors         : Maddie Masiello - mmasiell@calpoly.edu, Angel Soto
 * version         : 1.0
 * date            : 2025-04-19
 * compiler        : STM32CubeIDE v1.12.0
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2025 STMicroelectronics. All rights reserved.
 **************************************************************************
 * OVERVIEW:
```

```
 * This project is the source file for any delay implemented.
 ******************************************************************************/
/* USER CODE END Header */

#include "delay.h"

void SysTick_Init(void)
{
    SysTick->CTRL |= (SysTick_CTRL_ENABLE_Msk | SysTick_CTRL_CLKSOURCE_Msk);
    SysTick->CTRL |= SysTick_CTRL_TICKINT_Msk;
}

void delay_us(uint32_t us)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
    uint32_t start = DWT->CYCCNT;
    uint32_t ticks = us * (SystemCoreClock / 1000000);
    while ((DWT->CYCCNT - start) < ticks)
        ;
}
```

---

lcd.h

```
/* USER CODE BEGIN Header */
/******************************************************************************
 * EE 329 A4
 ******************************************************************************
 * @file           : lcd.h
 * @brief          :
 * project         : EE 329 S'25 Assignment A4
 * authors         : Maddie Masiello - mmasiell@calpoly.edu,
*                  :Angel Soto – asoto36@calpoly.edu
 * version         : 1.0
 * date            : 2025-04-19
 * compiler        : STM32CubeIDE v1.12.0
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2025 STMicroelectronics. All rights reserved.
 ******************************************************************************
 * OVERVIEW:
 * This project is the header file for the main lcd module.
 ******************************************************************************
/* USER CODE END Header */
#ifndef INC_LCD_H_
#define INC_LCD_H_

#include "stm32l4xx.h"
#include "delay.h"

#define LCD_PORT GPIOA
#define LCD_RS GPIO_PIN_4 // PA4
#define LCD_EN GPIO_PIN_5 // PA5
#define LCD_DB4 GPIO_PIN_0 // PA0
#define LCD_DB5 GPIO_PIN_1 // PA1
#define LCD_DB6 GPIO_PIN_2 // PA2
#define LCD_DB7 GPIO_PIN_3 // PA3
#define LCD_DATA_BITS (LCD_DB4 | LCD_DB5 | LCD_DB6 | LCD_DB7)
#define LCD_PORTEN RCC_AHB2ENR_GPIOAEN // port enable mask
```

```c
#define RSpos 4
#define ENpos 5
#define DB4pos 0
#define DB5pos 1
#define DB6pos 2
#define DB7pos 3


void LCD_init(void);
void LCD_command(uint8_t cmd);
void LCD_write_char(uint8_t letter);
void LCD_write_string(char *str);
void LCD_4b_command(uint8_t command);
void LCD_set_cursor(uint8_t pos[2]);
void LCD_clear(void);
void DisplayTime(uint8_t *digits);

#endif /* INC_LCD_H_ */
```

---

lcd.c

```c
#include "lcd.h"
/******************************************************************************
 * EE 329 A4 - D
 ******************************************************************************
 * @file           : main.h
 * @brief          : Reaction timer Game
 * project         : EE 329 S'25 Assignment A4 - Part D
 * authors         : Maddie Masiello - mmasiell@calpoly.edu
 *                          : Angel Soto - asoto36@calpoly.edu
 * version         : 1.0
 * date            : 2025-04-19
 * compiler        : STM32CubeIDE v1.12.0
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2025 STMicroelectronics. All rights reserved.
 ******************************************************************************
 * OVERVIEW:
 * This project is the source file for our lcd.
 ******************************************************************************/
/* USER CODE END Header */

void LCD_GPIO_Config(void)
{
    RCC->AHB2ENR |= (LCD_PORTEN); //Enable LCD GPIO Register
    LCD_PORT->MODER &= ~(0b11<<(2*DB4pos) |
    0b11<<(2*DB5pos) |
    0b11<<(2*DB6pos) |
    0b11<<(2*DB7pos) |
    0b11<<(2*RSpos) |
    0b11<<(2*ENpos)); //reset
LCD_PORT->MODER |= (0b01<<(2*DB4pos) |
    0b01<<(2*DB5pos) |
    0b01<<(2*DB6pos) |
    0b01<<(2*DB7pos) |
    0b01<<(2*RSpos) |
    0b01<<(2*ENpos)); //output
LCD_PORT->OTYPER &= ~(0b1<<(DB4pos) |
    0b1<<(DB5pos) |
```

```
    0b1<<(DB6pos) |
    0b1<<(DB7pos) |
    0b1<<(RSpos) |
    0b1<<(ENpos)); //psh/pull
LCD_PORT->OSPEEDR |= (0b11<<(2*DB4pos) |
    0b11<<(2*DB5pos) |
    0b11<<(2*DB6pos) |
    0b11<<(2*DB7pos) |
    0b11<<(2*RSpos) |
    0b11<<(2*ENpos));//vhi spd
LCD_PORT->PUPDR &= ~(0b11<<(2*DB4pos) |
    0b11<<(2*DB5pos) |
    0b11<<(2*DB6pos) |
    0b11<<(2*DB7pos) |
    0b11<<(2*RSpos) |
    0b11<<(2*ENpos)); //reset
LCD_PORT->PUPDR |= (0b00<<(2*DB4pos) |
    0b00<<(2*DB5pos) |
    0b00<<(2*DB6pos) |
    0b00<<(2*DB7pos) |
    0b00<<(2*RSpos) |
    0b00<<(2*ENpos)); //nopupl
}


void LCD_init(void)
{
    LCD_GPIO_Config(); // configure GPIO pins for LCD

    delay_us(40000); // power-up wait 40 ms
    for (int idx = 0; idx < 3; idx++)
    {                          // wake up 1,2,3: DATA = 0011 XXXX
        LCD_4b_command(0x30); // HI 4b of 8b cmd, low nibble = X
        delay_us(200);
    }
    LCD_4b_command(0x20); // fcn set #4: 4b cmd set 4b mode - next 0x28:2-line
    delay_us(40);         // remainder of LCD init removed - see LCD datasheets

    LCD_command(0x28); // Function set: 4-bit, 2-line, 5x8 font
    delay_us(300);

    LCD_command(0x10); // Set cursor move direction
    delay_us(300);

    LCD_command(0x0F); // Display ON, cursor ON, blink ON
    delay_us(300);

    LCD_command(0x06); // Entry mode: increment cursor, no shift
    delay_us(300);

    LCD_command(0x01); // Clear display
    delay_us(2000);    // Extra wait for clear command

    LCD_command(0x80); // Set cursor to home (DDRAM address 0)
    delay_us(300);
}

void LCD_pulse_ENA(void)
{
    // ENAble line sends command on falling edge
    // set to restore default then clear to trigger
```

```
    LCD_PORT->ODR |= (LCD_EN);  // ENABLE = HI
    delay_us(5);                // TDDR > 320 ns
    LCD_PORT->ODR &= ~(LCD_EN); // ENABLE = LOW
    delay_us(5);                // low values flakey, see A3:p.1
}

void LCD_4b_command(uint8_t command)
{
    // LCD command using high nibble only - used for 'wake-up' 0x30 commands
    LCD_PORT->ODR &= ~(LCD_DATA_BITS); // clear DATA bits
    LCD_PORT->ODR |= (command & 0x0F); // changed: place nibble directly on PA0-PA3
    delay_us(5);
    LCD_pulse_ENA();
}

void LCD_command(uint8_t command)
{
    // send command to LCD in 4-bit instruction mode
    // HIGH nibble then LOW nibble, timing sensitive
    LCD_PORT->ODR &= ~(LCD_DATA_BITS);                // isolate cmd bits
    LCD_PORT->ODR |= ((command >> 4) & LCD_DATA_BITS); // HIGH shifted low
    delay_us(5);
    LCD_pulse_ENA(); // latch HIGH NIBBLE

    LCD_PORT->ODR &= ~(LCD_DATA_BITS);          // isolate cmd bits
    LCD_PORT->ODR |= (command & LCD_DATA_BITS); // LOW nibble
    delay_us(5);
    LCD_pulse_ENA(); // latch LOW NIBBLE
}

void LCD_write_char(uint8_t letter)
{
    // calls LCD_command() w/char data; assumes all ctrl bits set LO in LCD_init()
    LCD_PORT->ODR |= (LCD_RS); // RS = HI for data to address
    delay_us(400);
    LCD_command(letter);        // character to print
    LCD_PORT->ODR &= ~(LCD_RS); // RS = LO
}

void LCD_write_string(char *str)
{
    // writes each character in a null-terminated string to LCD
    while (*str)
    {
        LCD_write_char(*str++);
    }
    delay_us(400);
}

void LCD_clear(void)
{
    // clears LCD display and sets cursor to home position
    LCD_command(0x01); // clear display command
    delay_us(10000);   // wait for clear command to complete
}

void LCD_set_cursor(uint8_t pos[2])
{
    // sets LCD cursor to given (col, row)
    uint8_t row_offsets[] = {0x00, 0x40};
    uint8_t addr = 0x80 | (pos[0] + row_offsets[pos[1]]);
```

```
    LCD_command(addr);
}

/*****************************************************************************
 *  DisplayTime(): Displays the current time on the LCD in MM:SS format
 * - Takes an array of 4 digits and displays them on the LCD.
 *****************************************************************************/
void DisplayTime(uint8_t *digits)
{ // points to the digits array

      // sets cursor to the appropriate position on the lcd and display the digits
      LCD_set_cursor((uint8_t[]){11, 1});
      LCD_write_char(digits[0] + '0');
      LCD_set_cursor((uint8_t[]){12, 1});
      LCD_write_char(digits[1] + '0');
      LCD_set_cursor((uint8_t[]){13, 1});
      LCD_write_char(':');
      LCD_set_cursor((uint8_t[]){14, 1});
      LCD_write_char(digits[2] + '0');
      LCD_set_cursor((uint8_t[]){15, 1});
      LCD_write_char(digits[3] + '0');
}
```